



INSTITUTO FEDERAL
Rio Grande do Norte

Desenvolvimento Sistemas Corporativos

Injeção de Dependência.

Professor Jeferson Queiroga - jeferson.queiroga@ifrn.edu.br



Motivação

- Sistemas reais possuem muitas classes que precisam umas das outras
- Quando uma classe cria suas próprias dependências, o código se torna difícil de alterar e testar
- A injeção de dependência (DI) ajuda a resolver isso de forma elegante



O que é uma dependência

- Uma **dependência** é qualquer objeto que uma classe precisa para funcionar corretamente.



```
public class PedidoService {  
    private EmailService emailService;  
}
```

Aqui, EmailService é uma **dependência** de PedidoService.



O problema do acoplamento

- Quando uma classe **cria diretamente** suas dependências, ocorre **acoplamento**:



```
public class PedidoService {  
    private EmailService emailService = new EmailService();  
}
```

Problemas:

- Dificulta **testes unitários**
- Dificulta **mudanças** (ex: trocar Email por SMS)
- Viola o princípio da **inversão de dependência (D - SOLID)**



Conceito de acoplamento

Qual tipo de acoplamento promove melhor flexibilidade e manutenção do código?



Alto Acoplamento

Cria código rígido e difícil de manter

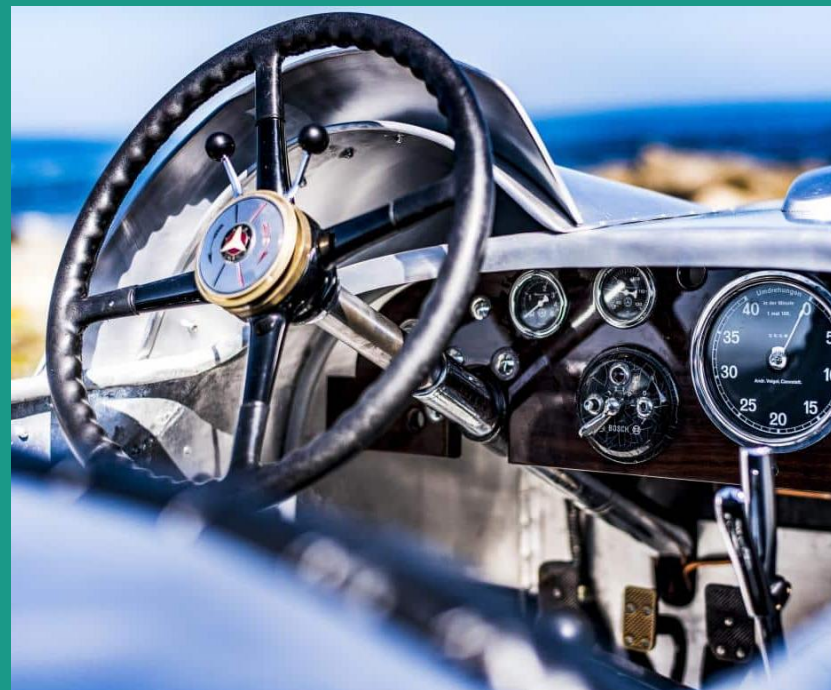


Baixo Acoplamento

Promove flexibilidade e facilidade de teste

Analogia prática

- Um carro acoplado seria aquele com o volante soldado nas rodas. Para trocar as rodas, teria que trocar o volante também.
- Um carro desacoplado usa engrenagens – você troca as rodas sem afetar o volante.



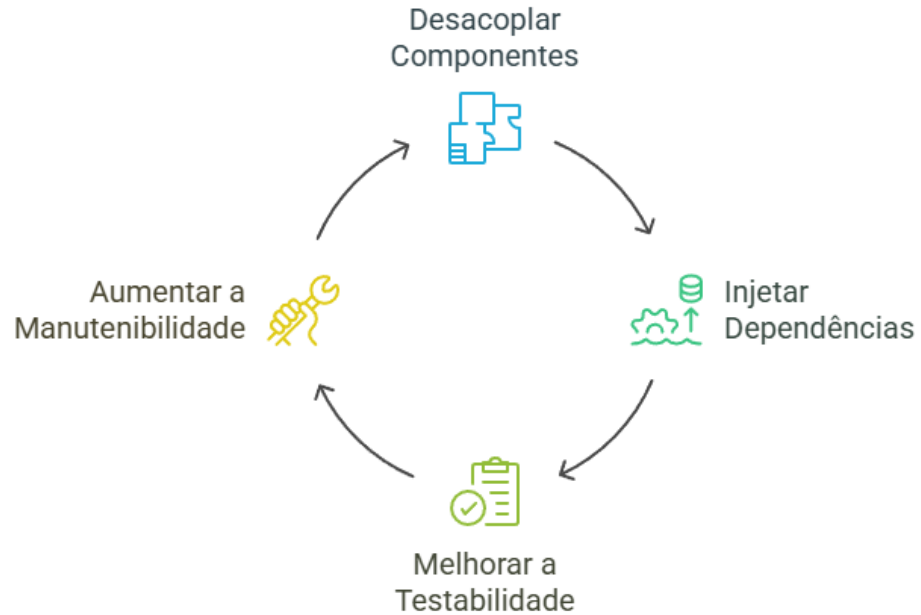


O que é Injeção de Dependência?

- Injeção de Dependência é um padrão de projeto usado para evitar o alto nível de acoplamento de código dentro de uma aplicação.
- Sistemas com baixo acoplamento de código são melhores pelos seguintes motivos:
 - Aumento na facilidade de manutenção/implementação de novas funcionalidades
 - Também habilita a utilização de mocks para realizar unit testes.



Ciclo de Injeção de Dependência





Sem injeção de dependência



```
public class PedidoService {  
    private EmailService emailService = new EmailService();  
  
    public void processar() {  
        emailService.enviarEmail("Pedido processado!");  
    }  
}
```

Problemas:

- Forte dependência de `EmailService`
- Se o meio de envio mudar, precisa alterar o código



Aplicando Injeção de Dependência

- Criamos uma interface para generalizar o comportamento
- Implementamos versões diferentes da dependência
- A classe principal recebe a dependência de fora



```
public interface NotificacaoService {  
    void enviar(String mensagem);  
}
```



Aplicando Injeção de Dependência



```
public class EmailService implements NotificacaoService {  
    public void enviar(String mensagem) {  
        System.out.println("E-mail: " + mensagem);  
    }  
}  
  
public class SmsService implements NotificacaoService {  
    public void enviar(String mensagem) {  
        System.out.println("SMS: " + mensagem);  
    }  
}
```



Classe principal com DI



```
public class PedidoService {  
    private final NotificacaoService notificacaoService;  
  
    // Injeção via construtor  
    public PedidoService(NotificacaoService notificacaoService) {  
        this.notificacaoService = notificacaoService;  
    }  
  
    public void processar() {  
        notificacaoService.enviar("Pedido processado!");  
    }  
}
```



Exemplo de uso

```
public class Main {  
    public static void main(String[] args) {  
        NotificacaoService notificacao = new EmailService();  
        PedidoService pedido = new PedidoService(notificacao);  
        pedido.processar();  
    }  
}
```

O objeto `PedidoService` **não sabe** como a mensagem é enviada — apenas **sabe que há um serviço que envia**.



O papel do Spring

- O **Spring Framework** automatiza a criação e a injeção de dependências por meio do seu **IoC Container**.
- **Sem o Spring:**
O desenvolvedor precisa **criar e gerenciar manualmente** os objetos.
- **Com o Spring:**
O **contêiner IoC** é responsável por **criar, gerenciar e injetar** os objetos automaticamente nas classes que precisam deles.



Como o Spring faz

- O Spring identifica as classes anotadas com **@Service**, **@Component**, **@Repository**, etc.
- Cria objetos (beans) automaticamente
- Injeta as dependências onde há **@Autowired**
- Você não precisa usar **new** — o Spring gerencia tudo.



Injeção por Construtor

- **Vantagens:** imutabilidade com final, fácil de testar, evita null.
- **Como:** recebe a dependência no construtor (o Spring injeta).

```
package com.exemplo.di.tipos;

import com.exemplo.di.core.NotificacaoService;
import org.springframework.stereotype.Service;

@Service
public class PedidoServiceConstrutor {

    private final NotificacaoService notificacaoService;

    // O Spring injeta aqui. Não precisa @Autowired em versões recentes.
    public PedidoServiceConstrutor(NotificacaoService notificacaoService) {
        this.notificacaoService = notificacaoService;
    }

    public void processar() {
        notificacaoService.enviar("Pedido processado (Construtor)!");
    }
}
```



Injeção por Campo



```
package com.exemplo.di.tipos;

import com.exemplo.di.core.NotificacaoService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class PedidoServiceCampo {

    @Autowired
    private NotificacaoService notificacaoService;

    public void processar() {
        notificacaoService.enviar("Pedido processado (Campo)!");
    }
}
```



Exemplo:

Problema: Fazer um programa para calcular o preço final de um produto, dado seu custo e o local de entrega.

Regras:

Taxa de entrega = R\$ 10,00 (RN) ou R\$ 20,00 (outros)

Imposto = 10%

Exemplo:

Custo: 300,00

Local de entrega: SC

Resposta = 350,00